

Local coding-agent benchmark Findings and practical picks

Measured September 18-21, 2026 (UTC). Report compiled 2026-09-21 02:19 UTC.

No configuration qualified as a verified coding-agent winner at 64K or in the completed 16K family queue. The picks below are useful starting points for supervised work or further investigation, with explicit limits.

Pick	Why I would keep it	Measured limit
1. Qwen3.6 27B i1-IQ2_M	Best observed coding quality; relatively few tool failures. Upstream for reliability, bundled for faster diagnostic fresh action.	28/36 coding passes. Long tasks: 4/8 completed; eliminated before qualification.
2. OxCoder 9B Q4_K_M / upstream	Best measured fast fallback with clean tool use across its completed coding tasks.	11/21 coding passes; 0/21 tool errors. Not qualified.
3. GPT-OSS 20B / interface research	Its basic native file tools worked. Patch-format incompatibility is a concrete issue worth isolating.	0/4 coding passes; all four failed the required patch format. Capacity parser also failed one seed.
4. Ternary Bonsai 2 27B / memory research	Exceptional weight-size reduction and demonstrated 64K capacity.	Slow fresh 64K response; 3/5 completed smoke passes, 2 tool failures. Stopped by user.

If I had to use a local assistant today, I would start with **Qwen3.6 IQ2_M** for work where correctness matters most, or **OxCoder 9B** when responsiveness and a functioning tool loop matter more. Both need tests and human review. Neither earns the original 64K agent label.

What changed during the search

Reducing context to 16K allowed some larger quantizations to fit, but it did not produce a qualifying agent. The new family queue finished cleanly. GPT-OSS hit the four-run tool-error cutoff; the other five models reached the regular coding elimination threshold.

Evidence scope: 1,093 recorded attempts and 63 registered configurations. The original ranking export contains 932 attempts and 50 configurations; 13 later configurations are covered separately. Attempt counts include retries, diagnostics, and interrupted records, not 1,093 distinct coding tasks.

What the benchmark actually required

Requirement	Original 64K contract	Separate 16K experiments
Context / fully templated input	65,536 / 61,440 tokens	16,384 / 12,288 tokens
Output reserve	4,096 tokens	4,096 tokens
Regular coding	At least 27/36, with complete coverage	Same frozen 27/36 gate
Context coding	At least 9/12 long tasks	Same 9/12 gate at 12,288-token target
Capacity	All nine marker values across three seeded requests	Same marker gate at the smaller input target
Final stability	20 valid fresh + 20 valid cached measurements; isolated demo	Same stability and demo requirements
Fit	Record real GPU and system memory	Full GPU layer offload, f16 KV, at least 512 MiB measured free headroom

Three speed metrics with different meanings

Fresh TTFT / first stream: time from request dispatch to the first model output event. That event can be reasoning. **First useful action:** time to a completely parsed, schema-valid code/tool action. This is the primary responsiveness measure. **Decode throughput:** output tokens divided by verified decode time; it does not include the full cost of processing a fresh long prompt.

Fresh measurements reset the model context and verify input/cache counters. Cached timing requires a verified stable-prefix priming procedure and is a separate secondary metric. Three diagnostic fresh samples do not establish the required 20-sample p95 or stability.

Tools and grading

Python and TypeScript fixtures use deterministic seeds, public tests, hidden checks, and gold implementations. Agents can inspect and modify only permitted source files. The tool interface requires native calls and the advertised argument schemas. The patch tool requires a numbered unified diff. Malformed or unpermitted calls fail that coding run.

The controller already allows iteration within a fixture: up to 16 tool turns, 32,768 generated tokens in aggregate, and a 900-second task deadline. Coding failures therefore measure the final task outcome after permitted interaction, rather than a single untested answer.

The latest family cutoff stops after four coding runs when two or more contain tool-call errors. That cutoff is separate from the regular early elimination threshold of ten failed coding tasks.

Original 64K search: quality beat headline speed

Model / engine	Coding pass/run	Tool errors per run	Fresh useful action (s)	Exact sustained tok/s
qwen36-27b-iq2 upstream	28/36	1/36	123.33-124.55	17.30
qwen36-27b-iq2 bundled-llama	28/36	2/36	110.75-112.83	17.13
qwen38-27b-iq3 upstream	15/25	10/25	116.60-120.82	15.79
oxcoder9b-q4km upstream	11/21	0/21	37.62-37.81	31.79
signofffour35ba3b-iq2m upstream	10/20	7/20	43.60-43.84	59.64
ornith15-9b-q4 bundled-llama	10/20	9/20	36.02-38.37	31.55

Coding denominators are completed valid fixture runs, not the 36-task target. Useful-action ranges have three screen samples per listed configuration. Sustained throughput is one exact full-context sample per listed configuration. None is a qualifying final p95.

Qwen3.6 IQ2_M: closest to the original quality gate

Both upstream and bundled engines passed **28/36 regular tasks (77.8%)** and all nine 64K markers. Long coding passed **4/8 completed tasks**; four failures made 9/12 impossible. Upstream failed the required single-final-action contract on all four failed long runs; bundled had three such failures and one timeout. This is a long-task tool-contract failure, not evidence that capacity exceeded GPU memory.

Upstream had **1 tool error in 36 coding runs**; bundled had **2 in 36**. Upstream fresh first-stream range was 119.54-119.68 seconds and useful action 123.33-124.55 seconds. Bundled was faster in its small screen: first stream 107.05-107.19, useful action 110.75-112.83 seconds.

OxCoder: a faster assistant with a weaker final solution rate

OxCoder 9B Q4_K_M on upstream passed **11/21 tasks (52.4%)**, with **zero tool errors in 21 completed runs**. Its fresh useful action was 37.62-37.81 seconds and sustained decode 31.79 tok/s. Ten deterministic test failures eliminated it before long coding and final stability.

Why I would not pick the throughput leader

SignOffFour 35B-A3B IQ2_M reached 59.64 tok/s, but passed only 10/20 completed tasks and had seven tool errors. Qwen3.8 IQ3_S passed 15/25, but all ten recorded coding failures were tool-stream failures. Those outcomes outweigh synthetic or decode speed under the stated priorities.

Completed 16K family queue

Installed model	Coding pass/run	All tool errors	First-four tool errors	Outcome
Devstral Small 2 24B Q3_K_L	2/12	3	0/4	Coding eliminated
Ministral 3 14B Reasoning Q4_K_M	4/14	3	0/4	Coding eliminated
GPT-OSS 20B Q4_K_S	0/4	4	4/4	Four-run stop
Gemma 4 12B QAT Q4_0	2/12	6	1/4	Coding eliminated
Gemma 4 E4B Q6_K	5/15	5	0/4	Coding eliminated
Nemotron 3 Nano 4B Q6_K	2/12	3	1/4	Coding eliminated

All-tool counts use completed valid regular coding records. A coding run is counted once if its final failure was a tool error, even when earlier native calls succeeded. Coding and tool-error counts overlap.

All six models fit the 16K memory gate. Devstral, Ministral, both Gemma models, and Nemotron passed all nine capacity markers. GPT-OSS passed the first two seeded capacity requests (six marker values); the third seed failed its native output parser.

The queue exited cleanly, with no active job or benchmark model server remaining. Devstral, Ministral, both Gemma models, and Nemotron accumulated ten regular task failures, making the 27/36 target mathematically unreachable. They did not proceed to long coding, final 20+20 timing, or the demo.

What these results do and do not mean

Gemma E4B had the best completed pass fraction in this small family queue, **5/15 (33.3%)**. That is still far below the required quality gate. The installed Nemotron tested here was **Nano 4B Q6_K**; this is not a verdict on larger Nemotron checkpoints. The GPT-OSS file was the requested **20B Q4_K_S** model.

The Gemma 12B test used exactly `F:/models/lmstudio-community/gemma-4-12B-it-QAT-GGUF/gemma-4-12B-it-QAT-Q4_0.gguf`. Muse/Glimmer was skipped at the user's request.

GPT-OSS needs a precise interpretation. Its four failed coding runs successfully listed and read source files, then sent Codex-style patch envelopes. The frozen tool required unified diffs and rejected those arguments. This is a tool-contract failure of the tested configuration; it is not evidence that every tool response was null.

Source: artifacts/families-16k/queue-status.json, per-model results.json and native turn logs; audit snapshots are embedded in this PDF.

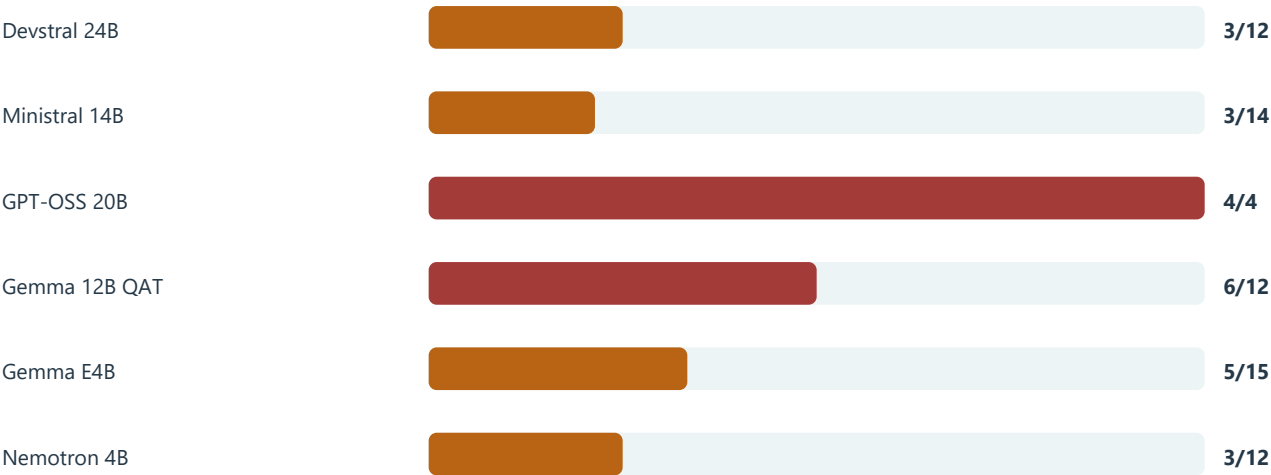
The fast 16K models still failed as agents

Model	Fresh TTFT median (s)	Fresh useful action median (s)	Native decode tok/s
Devstral Small 2 24B Q3_K_L	13.585	14.071	16.578
Ministral 3 14B Reasoning Q4_K_M	7.592	7.918	24.724
GPT-OSS 20B Q4_K_S	not measured	not measured	not measured
Gemma 4 12B QAT Q4_0	7.095	7.421	30.922
Gemma 4 E4B Q6_K	3.665	5.111	50.423
Nemotron 3 Nano 4B Q6_K	3.165	3.908	53.536

Fresh full-input action screens: three clean samples, 12,288 templated input tokens. Native throughput screens: one 256-output-token sample per shown result. These are structured-action diagnostics, not first source edits or the final 20-sample p95. GPT-OSS was stopped before these speed screens.

Gemma E4B and Nemotron reached approximately 50-54 output tok/s and single-digit fresh useful-action seconds at 16K. That speed was real for the diagnostic workload, but final coding pass rates and tool failures were too weak to recommend either as an unattended agent. Do not compare these smaller-prompt times directly with the 61,440-token original search.

Tool-error share across completed coding runs



Bar length is the fraction of completed coding runs ending in a malformed/unpermitted tool failure. The first-four cutoff used only the initial four runs, so later tool failures remain visible here. GPT-OSS errors were unsupported patch syntax after working native read tools.

Reliable tools make a feedback loop possible. These regular fixtures already gave models an opportunity to inspect files, patch code, run public tests, and revise within the bounded task budget. Low final coding accuracy cannot be dismissed as having had no chance to iterate.

The actual 16K memory boundary

Model / weight file	File size GB decimal	Measured free headroom MiB	16K verdict
Qwen3.8 27B installed IQ3_S	12.04	3,448	Fits; initial capacity probe passed
Qwen3.8 27B UD-Q4_K_M	16.46	290	Rejected; full-prompt request timed out
Qwen3.8 27B UD-Q4_K_S	15.36	364	Rejected; full-prompt request timed out
Qwen3.8 27B UD-IQ4_XS	14.25	1,354	Fits; all nine markers; user stopped coding
Qwen3.6 27B IQ4_XS	15.44	312	Rejected; full-prompt request timed out
Qwen3.6 27B UD-Q3_K_XL	14.47	1,164	Fits; all nine markers; 8/18 coding passes

The largest tested Qwen3.6 file that met the specific 16K fit rule was **UD-Q3_K_XL**: 8/18 coding passes, nine tool failures and one ordinary coding failure. The fitting Qwen3.8 IQ4_XS trial had 6/11 completed passes, four tool failures and one coding failure before the user stopped it. Neither result improves the evidence for a qualified agent.

Why disk size did not predict headroom

GGUF size is a disk representation. CUDA weight buffers, KV cache, recurrent state where applicable, compute buffers, runtime allocations, and desktop activity all contribute to residency. For Qwen3.6 IQ4_XS, the CUDA model buffer was about 14,032 MiB; at 16K its f16 KV cache added 1,024 MiB, plus about 150 MiB recurrent state and 138 MiB compute buffer.

For Qwen3.6 Q3_K_XL, the CUDA weight buffer fell to about 13,110 MiB. That reduction restored sufficient measured headroom. A smaller file can still sit near the memory limit if the remaining runtime allocations consume the difference.

The reserved-memory correction

NVML reported 16,380 MiB total, but used plus free equaled about 16,110 MiB, leaving 270 MiB reserved. Fit now subtracts that reserve and uses the smaller of reported free memory and available memory minus peak usage. The earlier total-minus-used calculation overstated headroom.

Headroom is measured for the recorded probe and background workload. It is not a promise that a changing desktop workload cannot consume it. Shared WDDM allocation is reported as a diagnostic delta; it alone does not prove weight spill or paging.

Shorter context helps fit and fresh prefill

Context is measured in tokens, not kilobytes. The configured window must hold fully templated input and output together. The benchmark uses 61,440 + 4,096 at 64K, or 12,288 + 4,096 at 16K.

There are two useful effects

More room for weights: a smaller context window reduces the KV footprint for these settings, which can enable a higher quantization or additional draft weights to remain on the GPU. **Less fresh input to process:** a genuinely smaller prompt also reduces prefill work. The second benefit exists even when the model weights and quantization remain unchanged.

Merely changing the configured maximum while sending the same short prompt does not establish a fresh-latency improvement. Conversely, a model can fit a large configured window and still take a long time to process a nearly full, uncached input.

The measured counterexample to file-size intuition

A roughly 10.00 GB Qwen3.6 IQ2_M file could load at 64K with f16 KV and pass capacity. A 15.44 GB Qwen3.6 IQ4_XS file failed the 16K headroom rule. That is consistent: the higher-precision weights consumed the memory recovered from the smaller cache, along with runtime overhead.

Higher quantization did not establish higher agent quality

The new 16K trials changed both the context regime and weight representation. They were not a randomized identical-settings quality comparison. Their measured result is narrower: the fitting Qwen3.6 Q3_K_XL stack failed the regular coding gate, and the Qwen3.8 IQ4_XS stack was stopped before full coverage. There is no measured crossover where those tested stacks became qualified coding agents.

Capacity is not long-context coding

Retrieving nine markers confirms that the prompt reached the model and it can extract the specified evidence. Long coding adds reasoning about distributed source facts and producing a correct final action. Qwen3.6 IQ2_M passed retrieval capacity yet failed enough long coding tasks to eliminate it.

The user explicitly authorized separate 16K experiments. The original 64K grading contract remains intact, and no 16K result is relabeled as a 64K success.

Ternary Bonsai: real memory result, limited agent evidence

Observation	Measured result
Checkpoint / representation	prism-ml/Ternary-Bonsai-2-27B-gguf, PTQ1_0
Downloaded weight size	5,946,648,928 bytes (5.95 GB decimal)
Required runtime	Prism ternary fork b10709, commit 9a9394a895b96003ca842a6041cb28ac49a108f7
GPU / context	All 65 layers offloaded; configured 65,536 tokens
64K capacity	Three fresh full-prompt requests; all nine marker values passed
Measured peak GPU use	10,426-10,554 MiB in full-context capacity records
Coding smoke	3/5 completed passes; 2 malformed/unpermitted tool-stream failures
Stop status	Stopped by user; sixth smoke record interrupted, not a completed failure

Fresh input	First model output (s)	First useful action (s)	Decode tok/s
16,384 tokens / small probe	40.665	54.089	27.487
61,440 tokens / 64K, n=3	167.475-169.217	187.510-192.085	21.461-21.477

Both Bonsai rows used a configured 65,536-token window; the small probe had 16,384 input tokens. These are capacity-request measurements, not the regular timing workload or a qualifying sustained decode comparison. No cached latency was measured. The three full-context capacity intervals had no flagged foreign GPU overlap.

Bonsai delivered an unusually small weight file and comfortable memory residency at 64K. It did **not** deliver a fast fresh useful action on this host: nearly full uncached input needed about three minutes before the valid capacity action. The regular coding, long coding, stability, and demo contracts were not completed.

I would retain it as a memory-efficiency research candidate, especially if a compatible tool/template configuration can be tested. I would not choose it as the everyday coding agent based on this evidence.

Pinned model revision: 6ed5e12bf84b7a63069882c91dd9e9218647d17b. Weight SHA256: 53107f530aa52eb00912263ab1ee29bd199261c87cd7b4ad4ca1318c1fe33ee3. The primary native llama.cpp engine was not substituted for its required custom kernel support.

MTP and DFlash: real drafting, no qualifying improvement

Target KV	Mode	64K markers	Clean capacity timings	Coding smoke	Valid fresh timings
q8_0 / q8_0	Off	9/9	3/3	4/6	3
q8_0 / q8_0	MTP	9/9	1/3	3/6	0
q8_0 / q8_0	DFlash	9/9	0/3	5/6	0
q4_0 / q4_0	Off	9/9	3/3	3/6	3
q4_0 / q4_0	MTP	9/9	2/3	3/6	0
q4_0 / q4_0	DFlash	9/9	1/3	3/6	0

The target was Qwen3.6 27B IQ2_M. Native counters confirmed real drafting: **MTP accepted 90.4-92.1%** of drafted tokens across capacity records; **DFlash accepted 58.4-58.9%**. MTP used 1.680 GB Q4_0 heads and width two. DFlash used a 1.849 GB Q8_0 drafter and comparison width seven. Draft cache was q8_0/q8_0.

Clean auxiliary capacity decoding reached **24.22-25.49 tok/s with MTP** and **36.71 tok/s for one DFlash q4-cache sample**. This is a promising decode signal with small sample counts, not a demonstrated fresh-action improvement or coding qualification.

All six arms passed retrieval capacity. None of the speculative arms passed its complete screen. Foreign GPU overlap made many speculative timing intervals diagnostic only; those values are excluded from clean speed comparisons. The arms also failed required coding/action screening gates, so a capacity pass cannot establish a usable agent.

What I conclude

Speculation remains a plausible decoding optimization, but this run did not verify that it improves the desired fresh useful-action metric. It adds weight and KV memory pressure, and the model still has to process a long fresh prompt before decoding. A higher output tok/s value cannot by itself rescue delayed or invalid tools.

What I would test if this search continues

First select a target configuration with reliable tools and adequate quality. Then compare off versus MTP versus DFlash under the same target weights, prompt, sampling, and quiet GPU conditions, preserving native acceptance counters and memory measurements. Only surviving arms should receive full regular/long qualification and final 20+20 timing.

Source: artifacts/speculation-measured-results.json/.md and speculation-comparison-plan.json. The report labels these as screening results, not full coding qualification. Exact arm settings, counters, attempt IDs, and launch arguments are retained in the embedded focused audit and local raw export.

Engine coverage and barriers

Engine / version	Observed local outcome	What remains unproven
Upstream llama.cpp b11040 5b335f413e4f...	Ran the strongest Qwen3.6 baseline, OxCoder, speculative arms, and all six new 16K family models.	No model passed the complete quality/stability contract.
Bundled llama.cpp b10472 7a556b8f9...	Matched Qwen3.6 regular quality; faster three-sample fresh-action screen.	No full-context coding winner or final p95.
CUDA TurboQuant fork 8ed935a092ee...	Isolated CUDA runtime exercised; compressed-cache and control profiles entered the registry.	No complete qualifying tool/coding/long/stability result.
ik_llama.cpp	Container preflight failed to load libcuda.so.1.	No comparable successful model benchmark.
ExLlamaV3 / TabbyAPI	ExLlamaV3 1.5.0+cu128.torch2.9.0; Tabby 53da7919d4e...; model startup lacked preprocessor_config.json.	Performance and quality are unavailable for that stack.
vLLM v0.29.0	Recorded Docker storage headroom barrier; probe budget later exhausted.	No successful comparable local endpoint measurement.
SGLang v0.5.19	Recorded Docker storage headroom barrier; probe budget later exhausted.	No successful comparable local endpoint measurement.
LM Studio	Authenticated comparison was attempted; a pending load could not be confirmed complete. Later probe budget exhausted.	No qualified LM Studio measurement; not an authentication-unavailable label.
Ollama	Comparison encountered unresolved owned managed-load state; probe budget later exhausted.	No qualifying comparative agent result.
Prism ternary fork b10709 / 9a9394a895...	Ran Bonsai full-offload 64K capacity and five completed coding smokes.	User stopped before qualification.

Many earlier attempts were delayed by an unresolved benchmark-owned managed load. The controller refused to start another GPU workload while ownership was uncertain. Those availability records do not make an engine a measured performance loser.

Discovery versions are pinned historical leads, not a claim that each engine's newest release was installed or fully benchmarked. Exact binary, package, source and container pins are recorded in the source artifacts. No driver, global environment, WSL, clocks, or personal repositories were changed to force a comparison.

Failure diagnosis and confidence limits

GPT-OSS: patch dialect and a separate native parser issue

The first four GPT-OSS coding runs emitted native `list_files` and `read_file` calls successfully. The next `apply_patch` calls supplied JSON-valid arguments containing Codex-style `*** Begin Patch / *** Update File` envelopes. The frozen executor requires `-- / +++` numbered unified hunks and rejected that format.

In a separate capacity request, the native Harmony parser failed to parse a final constrained-tool output. Two earlier seeded requests succeeded. These are concrete integration failures in the tested stack. An isolated prompt/template/parser experiment could be useful, but must rerun the same fixtures and grading rules; it cannot silently credit unsupported patches as passes.

Missing counters were not necessarily tokenizer defects

Earlier Devstral and Ministral labels such as `small_context_token_count_mismatch` followed requests that timed out without final token counters. GPT-OSS also had an engine error that ended the stream before usage appeared. The label describes failed evidence validation; the underlying error is needed to diagnose the cause. Both Devstral and Ministral later passed all nine capacity markers at 16K.

Historical harness issues were identified and corrected

A directory spelling bug rejected native `list_files` arguments containing `src/` while accepting `src`. The source-directory validation was repaired under a new protocol identity, and subsequent tests preserved traversal restrictions. Historical records remain visible rather than being rewritten as successes.

A prolonged Qwen3.5 reasoning stream was investigated and found to be actively producing reasoning, rather than a stalled HTTP read or demonstrated OOM. Fast first reasoning did not mean fast first useful code. Interrupted attempts remain incomplete, and raw byte/event counts were not converted into token throughput.

Limits that materially affect the recommendations

No candidate reached the complete 20 fresh + 20 cached + demo qualification sequence. Many engines were unavailable or bounded out. Some coding runs were stopped early by mathematical elimination, others by the user. Quantization/context comparisons changed more than one setting. GPU overlap and incomplete attribution make flagged timing diagnostic. This is a finite local search, not a universal model leaderboard.

Several older focused `results.md` files retain a generic Qwen3.8 heading even for Qwen3.6 wrappers. This PDF uses JSON configuration identities and verified model paths, not those inherited headings.

My picks and the next useful experiment

1. Qwen3.6 27B IQ2_M: quality-first supervised baseline

It is the only tested baseline in this search that completed and passed the regular 27/36 threshold: 28/36 on both native engines. Its upstream tool-error rate was 1/36. I would retain upstream as the reliability reference and bundled as the faster candidate from the small fresh screen. The speed difference is not a final p95 ranking. Its failed long coding means I would keep supplied repository context focused and verify changes with tests.

2. OxCoder 9B Q4_K_M / upstream: faster, repairable loop

Zero tool errors in 21 completed coding tasks is especially relevant to the stated preference that broken tools are worse than wrong code. Its 37.62-37.81-second fresh full-context useful-action screen and 31.79 tok/s sustained sample were substantially more responsive than the large dense Qwen baseline. It still failed ten deterministic coding tasks. I would use it for supervised small edits and debugging, not unattended acceptance of changes.

3. GPT-OSS 20B: best concrete interface-repair lead

The tested configuration failed the four-run rule and should remain disqualified. Still, the saved calls point to a specific patch-format mismatch after working file tools, rather than an absence of native tool ability. A pinned configuration that clearly follows the advertised unified-diff contract, with compatible native output parsing, deserves a small four-run retest before spending another full sweep.

4. Bonsai: keep for memory research, not responsiveness

Its 5.95 GB file and demonstrated 64K GPU residency make it informative. Its nearly three-minute fresh full-context useful action and two tool failures in five completed smokes make it a poor current everyday coding pick. Better compression was real; a better agent was not established.

What I would pause or deprioritize

I would not repeat the same failing configurations. Qwen3.8 had repeated malformed/unpermitted calls across tested quantizations. The new family models failed regular quality well before long/stability testing. MTP and DFlash should wait for a reliable target and quiet timing intervals. Unavailable engines need a concrete barrier fix before another performance comparison.

Highest-value continuation: a bounded GPT-OSS interface retest, or an iteration-focused OxCoder experiment, using the same hidden tests and an explicit new configuration identity. Broader engine and model discovery remains possible within the shared budgets, but there is no measured justification for restarting the whole search.

Reproducing the two practical baselines

These are recorded native launch settings, not a claim that an agent endpoint is currently running or qualified. Native model servers expose inference; the benchmark controller supplies tool execution, workspace restrictions, and grading.

Qwen3.6 IQ2_M / upstream b11040

```
$model = "F:/models/mradermacher/Qwen3.6-27B-i1-GGUF/Qwen3.6-27B.i1-IQ2_M.gguf"
$engineArgs = @(
    "--model", $model, "--ctx-size", "65536", "--parallel", "1",
    "--n-gpu-layers", "99", "--flash-attn", "on",
    "--cache-type-k", "f16", "--cache-type-v", "f16",
    "--batch-size", "2048", "--ubatch-size", "512",
    "--threads", "16", "--threads-batch", "16",
    "--host", "127.0.0.1", "--port", "38201", "--jinja",
    "--no-context-shift", "--device", "CUDA0", "--verbosity", "4",
    "--cors-origins", "http://127.0.0.1:38201",
    "--spec-type", "none", "--metrics", "--slots"
)
& "F:/local-llm-benchmark-runtime/upstream-5b335f413e4f73b0809c4fe39af894efbcc6a0d2/bin/llama-server.exe" @engineArgs
```

Long filesystem lines in the recipe may wrap in the viewer; the exact recorded argv array is embedded in original-audit.json. Launch only one model on the GPU and confirm the port is free.

OxCoder 9B Q4_K_M / the same upstream engine

Use its recorded model file below with the same displayed upstream context/cache/server settings; the embedded audit retains the full exact argv for this configuration.

```
E:/modelmadness/prithivMLmods--OxCoder-9B-GGUF/d108751ef79e645b07d04030427b6b67b284d07e/OxCoder-9B.Q4_K_M.gguf
```

Request sampling for both references: temperature 0.6, top_p 0.95, top_k 20, min_p 0, presence_penalty 0, repeat_penalty 1, with recorded deterministic seeds. Reasoning mode default; speculative decoding off.

Qwen upstream config: 05c146133e492ac2e0df5f3637ecee19c2b9d797d579431a2321d54c5f91e46f. Model SHA256: 0a2fa2d3fc86a49dc69c7ab0306cfc954bca136a3bfa720165fd2c7de429f250.

OxCoder config: 3109dec088698c4578f570e38e0657f5ed7f2a340a1c202e941086c5b29bd36d. Model SHA256: a171c091c67c3547c45ec58f1b85a888d73725315fde71bc929ff9f8c255bb6f.

Host, budgets, provenance, and raw evidence

Item	Recorded value
GPU / driver	NVIDIA RTX 4060 Ti 16 GB; driver 591.86; total 16,380 MiB; available used+free about 16,110 MiB
CPU / memory	AMD Threadripper 1950X, 16 cores / 32 threads; 128 GiB system RAM
Execution clock	First runtime probe: 2026-09-18 17:46:39 UTC. Immutable deadline: 2026-09-25 17:46:39 UTC. 168-hour elapsed cap.
New model weight ledger	166,804,714,750 bytes across 23 acquired reservations. 133,195,285,250 bytes remain below the 300 GB decimal cap.
Downloads / single GPU	User-selected E:/modelmadness; installed files reused before new weights. GPU trials sequential.
Harness verification	Original speculation harness: 588 tests, one skip. Expanded 16K parent: 597 tests, one skip; focused wrapper scope checks passed.
Source snapshot	Private GitHub repository barretts/local-llm-benchmark; initial source commit 6df0b36044255269656bcaaa1bcc615a1923c128.
End of queue	All six family workers exit code 0; no active job; no benchmark model server. Final observed GPU baseline about 435 MiB at 33 C.

Primary local evidence

Original search: artifacts/ranking.json, recommendation.md, results.json, results.csv and report.html. Original export cutoff: 2026-09-20T18:26:49.860480+00:00.

Live durable record: state/benchmark.sqlite3. The report read it with a read-only SQLite connection; it did not restart or change the benchmark.

16K families: artifacts/families-16k/queue-status.json and each model's results.json, selection.json, plan.json and worker logs.

Weight fit trials: artifacts/qwen38-16k/, qwen36-16k/, and qwen36-q3kxl-16k/. **Bonsai:** artifacts/bonsai-2-27b/ plus its saved database configuration.

Speculation: artifacts/speculation-measured-results.json and speculation-comparison-plan.json. **Engine probes:** engine-manifest.json, engine-discovery.json and runtime/container pin files.

Raw streams and grading: .logs/ and the local fixture/grader artifacts. These retain actual usage counters, SSE events, execution outcomes, resource samples and exact launch metadata.

Two compact JSON audit snapshots are attached to this PDF. They include configuration identities, versions, hashes, launch argv, per-run outcomes, and sample counts. Raw multi-megabyte streams and the complete database remain local. Credentials are excluded.

External Reddit speed reports were discovery leads only. Different GPUs, short inputs, cache restoration, or decode-only claims do not establish this machine's fresh full-context useful-action latency. No external headline speed is used to choose a winner in this report.

Original configuration inventory

ID / model	Engine / KV profile	Coding T=tool errors	Long pass/run	Capacity requests	tok/s
05c14613 qwen36-27b-iq2	upstream f16/f16	28/36 T=1	4/8	3/3	17.30
79de48fe qwen36-27b-iq2	bundled f16/f16	28/36 T=2	4/8	3/3	17.13
43393b40 qwen38-27b-iq3	upstream f16/f16	15/25 T=10	0/0	3/3	15.79
3109dec0 oxcoder9b-q4km	upstream f16/f16	11/21 T=0	0/0	3/3	31.79
2d4018d1 signoffour35ba3b-iq2m	upstream f16/f16	10/20 T=7	0/0	3/3	59.64
37b148db ornith15-9b-q4	bundled f16/f16	10/20 T=9	0/0	3/3	31.55
e3a5bf35 oxcoder9b-q4km	bundled f16/f16	9/19 T=2	0/0	3/3	31.68
aa30f2d0 qwen35-35ba3b-iq3	upstream f16/f16	8/18 T=2	0/0	3/3	52.97
043b9421 gemma4-12b-qat-q4	bundled f16/f16 reasoning reduced	7/17 T=7	0/0	3/3	not measured
3ff888e9 qwen35-9b-udq5	upstream f16/f16	6/16 T=2	0/0	3/3	27.89
46877115 ornith15-9b-q4	TurboQuant f16/f16	6/16 T=6	0/0	3/3	31.43
478b8ec9 qwen35-35ba3b-autoround-q2	bundled f16/f16	6/16 T=4	0/0	3/3	58.97
365cd0a9 signoffour35ba3b-iq2m	bundled f16/f16	5/15 T=8	0/0	3/3	59.97

All rows are disqualified. Coding and long denominators are completed valid tasks; 0/0 means unmeasured, not zero accuracy. Capacity requests require three marker values each: 3/3 requests is all nine values. Throughput is only shown when the original report retained an exact sustained full-context sample. Reasoning and speculation variations are distinct configurations.

The complete 64-character IDs, exact settings, failure reasons, versions, hashes, sample counts and argv are in the attached original audit. The 13 later registered configurations are covered in the main report and focused audit. This inventory is not sorted as a qualified ranking.

Original configuration inventory

ID / model	Engine / KV profile	Coding T=tool errors	Long pass/run	Capacity requests	tok/s
e351b070 qwen36-27b-iq2	upstream q8_0/q8_0 draft-dflash	5/6 T=0	0/0	3/3	not measured
09b315f7 qwen36-27b-iq2	upstream q8_0/q8_0	4/6 T=0	0/0	3/3	not measured
18c70cef gemma4-e4b-q6	upstream f16/f16	4/14 T=3	0/0	3/3	37.75
5d8b4b2d ornith15-9b-q4	TurboQuant tbqp3/tbq3	4/14 T=7	0/0	3/3	not measured
3a1a7bb7 qwen35-9b-q4-plain	TurboQuant f16/f16	3/13 T=3	0/0	3/3	31.22
5b7e2072 qwen36-27b-iq2	upstream q4_0/q4_0 draft-dflash	3/6 T=1	0/0	3/3	not measured
65d757e3 qwen36-27b-iq2	upstream q4_0/q4_0 draft-mtp	3/6 T=1	0/0	3/3	not measured
6a865660 ornith10-35b-iq2	upstream f16/f16	3/13 T=7	0/0	3/3	56.99
90786423 qwen36-27b-iq2	upstream q8_0/q8_0 draft-mtp	3/6 T=0	0/0	3/3	not measured
918a55d7 qwen36-27b-iq2	upstream q4_0/q4_0	3/6 T=1	0/0	3/3	not measured
9ea92627 qwen35-4b-udq5	upstream f16/f16	3/13 T=2	0/0	3/3	42.56
c5fea732 qwen35-9b-q4-plain	upstream f16/f16	3/13 T=3	0/0	3/3	31.36
faa9edc1 qwen35-4b-udq5	bundled f16/f16	3/13 T=2	0/0	3/3	42.89

All rows are disqualified. Coding and long denominators are completed valid tasks; 0/0 means unmeasured, not zero accuracy. Capacity requests require three marker values each: 3/3 requests is all nine values. Throughput is only shown when the original report retained an exact sustained full-context sample. Reasoning and speculation variations are distinct configurations.

The complete 64-character IDs, exact settings, failure reasons, versions, hashes, sample counts and argv are in the attached original audit. The 13 later registered configurations are covered in the main report and focused audit. This inventory is not sorted as a qualified ranking.

Original configuration inventory

ID / model	Engine / KV profile	Coding T=tool errors	Long pass/run	Capacity requests	tok/s
fb87e4f qwen35-9b-udq5	bundled f16/f16	3/13 T=1	0/0	3/3	27.73
6bd4f295 nemotron3nano-4b-q6	bundled f16/f16	2/12 T=3	0/0	3/3	47.74
7228e67d qwen35-9b-q4-plain	TurboQuant tbqp3/tbq3	2/12 T=3	0/0	3/3	not measured
cc85554c qwen35-35ba3b-autoround-q2	upstream f16/f16	2/12 T=6	0/0	3/3	58.22
a412d3c7 nemotron3nano-4b-q6	upstream f16/f16	1/12 T=3	0/0	3/3	46.87
ba7d374c ornith15-9b-q4	upstream f16/f16	1/12 T=8	0/0	3/3	31.71
005751f7 gemma4-e4b-q6	bundled f16/f16	0/0 T=0	0/0	2/3	not measured
05972e93 gptoss20b-q4ks	upstream f16/f16	0/0 T=0	0/0	0/3	not measured
08ab659f qwen36-12b-heretic-iq4	upstream f16/f16	0/0 T=0	0/0	0/3	not measured
13f52847 gemma4-12b-qat-q4	upstream f16/f16	0/0 T=0	0/0	2/3	not measured
14dbcc62 minstral3-14b-reasoning-q4	bundled f16/f16	0/0 T=0	0/0	0/3	not measured
1f5873b1 granite41-8b-udq4	bundled f16/f16	0/0 T=0	0/0	0/3	not measured
361dd37e qwen36-35ba3b-heretic-iq2	upstream f16/f16	0/0 T=0	0/0	0/3	not measured

All rows are disqualified. Coding and long denominators are completed valid tasks; 0/0 means unmeasured, not zero accuracy. Capacity requests require three marker values each: 3/3 requests is all nine values. Throughput is only shown when the original report retained an exact sustained full-context sample. Reasoning and speculation variations are distinct configurations.

The complete 64-character IDs, exact settings, failure reasons, versions, hashes, sample counts and argv are in the attached original audit. The 13 later registered configurations are covered in the main report and focused audit. This inventory is not sorted as a qualified ranking.

Original configuration inventory

ID / model	Engine / KV profile	Coding T=tool errors	Long pass/run	Capacity requests	tok/s
6a52b282 qwen36-35ba3b-heretic-iq2	bundled f16/f16	0/0 T=0	0/0	0/3	not measured
7fad4a90 devstral-small2-24b-q3	upstream f16/f16	0/0 T=0	0/0	0/3	not measured
82fb4105 gemma4-12b-qat-q4	bundled f16/f16	0/0 T=0	0/0	2/3	not measured
8f6c1ea6 qwen36-27b-iq2	bundled q8_0/q8_0	0/0 T=0	0/0	2/3	not measured
9204508d qwen36-12b-heretic-iq4	bundled f16/f16	0/0 T=0	0/0	0/3	not measured
aeaa4cd2 qwen35-9b-udq5	bundled f16/f16	0/0 T=0	0/0	0/3	not measured
aecbbe5f qwen35-9b-q4-plain	bundled f16/f16	0/0 T=0	0/0	0/3	not measured
bdc7c79f gemma4-12b-qat-q4	upstream f16/f16 reasoning reduced	0/0 T=0	0/0	1/3	not measured
cf66425b granite41-8b-udq4	upstream f16/f16	0/0 T=0	0/0	2/3	not measured
d1ddf98c ministral3-14b-reasoning-q4	upstream f16/f16	0/0 T=0	0/0	0/3	not measured
f9144256 gptoss20b-q4ks	bundled f16/f16	0/0 T=0	0/0	2/3	not measured

All rows are disqualified. Coding and long denominators are completed valid tasks; 0/0 means unmeasured, not zero accuracy. Capacity requests require three marker values each: 3/3 requests is all nine values. Throughput is only shown when the original report retained an exact sustained full-context sample. Reasoning and speculation variations are distinct configurations.

The complete 64-character IDs, exact settings, failure reasons, versions, hashes, sample counts and argv are in the attached original audit. The 13 later registered configurations are covered in the main report and focused audit. This inventory is not sorted as a qualified ranking.